LangChain & LangGraph

Weeks 5 & 6 -- Day-by-Day Study Plan

Week 5: LangGraph Advanced		Week 6: Real Projects
Day 29	Multi-Agent Systems -- supervisor pattern, subgraphs, specialist agents	
Day 30	Subgraphs & Complex State -- MessagesState, add_messages, reducers	
Day 31	Parallelism in LangGraph -- fan-out/fan-in, map-reduce patterns	
Day 32	Error Handling & Retries -- retry_policy, fallback nodes, recursion limit	
Day 33	LangGraph Studio & Visualization -- live graph UI, Mermaid, LangSmith	
Day 34-35	Week 5 Project -- Multi-Agent Research System with async streaming	
Day 36-38	Project 1 -- AI Customer Support Bot with routing & escalation	
Day 39-42	Project 2 -- Coding Assistant Agent with plan-execute-reflect	

PREREQUISITES

Before starting Week 5, confirm you have completed:

Week 3: Tools, AgentExecutor, LangSmith, advanced retrieval, routing

Week 4: StateGraph, conditional edges, ReAct agent, checkpointing, interrupts

A working SqliteSaver-backed stateful agent from Days 27-28

langgraph, langchain-openai, langsmith, tavily-python all installed

Week 5 -- LangGraph Advanced

Day 29 . Multi-Agent Systems -- Supervisor Pattern

(1) THEORY -- TOPICS TO COVER

Why multi-agent: specialisation, parallelism, separation of concerns
Supervisor pattern: one orchestrator LLM routes to specialist agents
Each specialist is a ReAct agent with its own tools and system prompt
Subgraph pattern: compiled graphs nested as nodes in a parent graph
Shared vs isolated state between supervisor and sub-agents
When the supervisor decides the overall task is complete

(2) FOCUS / SKIP GUIDE

FOCUS ON

The supervisor node: takes current task, decides which agent to call next
How each sub-agent is just a create_react_agent compiled and wrapped as a node
State design: what fields does the supervisor need vs what do sub-agents need
The FINISH signal: supervisor returns 'FINISH' when all work is done
Testing with a multi-step task that genuinely requires 2+ specialist agents

DON'T WORRY ABOUT YET

Network-based multi-agent communication over HTTP -- local graphs only for now
Agent memory isolation and state merging strategies at scale
LangGraph Platform deployment with multiple agents -- Weeks 7-8
Custom supervisor LLMs with fine-tuned routing -- use gpt-4o-mini for now

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] LangGraph multi-agent tutorial (official)
-> https://langchain-ai.github.io/langgraph/tutorials/multi_agent/multi_agent_collaboration/
[Docs] LangGraph -- Supervisor agent how-to
-> https://langchain-ai.github.io/langgraph/tutorials/multi_agent/agent_supervisor/
[YouTube] LangChain -- Multi-agent systems tutorial
-> https://www.youtube.com/watch?v=hvAPnpSfSGo
[Docs] LangGraph -- Hierarchical agent teams
-> https://langchain-ai.github.io/langgraph/tutorials/multi_agent/hierarchical_agent_teams/
[Blog] LangChain blog -- Multi-agent workflows
-> https://blog.langchain.dev/langgraph-multi-agent-workflows/

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] Build 3 specialist agents: researcher (Tavily), coder (Python REPL), writer (summariser)
- [ ] Build supervisor node: routes to researcher/coder/writer or returns FINISH
- [ ] Connect supervisor to sub-agents via conditional edges
- [ ] Test: 'Write a Python script to fetch and display top 5 news headlines'
- [ ] Trace the run -- count how many agent hops the supervisor made
- [ ] Test with a task that needs researcher then writer (2 hops)

TASK RESOURCES

[Docs] LangGraph -- Agent supervisor notebook
-> https://langchain-ai.github.io/langgraph/tutorials/multi_agent/agent_supervisor/
[Docs] LangChain -- PythonREPLTool
-> https://python.langchain.com/docs/integrations/tools/python/
[Docs] LangGraph -- How to pass private state between agents
-> https://langchain-ai.github.io/langgraph/how-tos/pass_private_state/

(4) CODE EXAMPLES

Supervisor + specialist agents

```
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.prebuilt import create_react_agent
from langchain_community.tools.tavily_search import TavilySearchResults
from langchain_core.messages import HumanMessage
from typing import Literal

# Build specialist agents
researcher = create_react_agent(llm, tools=[TavilySearchResults(max_results=3)])
coder      = create_react_agent(llm, tools=[python_repl_tool])
writer     = create_react_agent(llm, tools=[])

# Wrap each as a node function
def researcher_node(state): return researcher.invoke(state)
def coder_node(state):      return coder.invoke(state)
def writer_node(state):     return writer.invoke(state)

# Supervisor: decides who goes next
MEMBERS = ['researcher', 'coder', 'writer']
supervisor_prompt = f'''You are a supervisor managing: {MEMBERS}.
Given the conversation, decide who should act next.
Reply ONLY with one of: {MEMBERS + ["FINISH"]}'''

def supervisor_node(state: MessagesState) -> dict:
    msgs = [{'role': 'system', 'content': supervisor_prompt}] + state['messages']
    resp = llm.invoke(msgs)
    return {'messages': [resp], 'next': resp.content.strip()}

from typing import TypedDict
class SupervisorState(MessagesState):
    next: str

def route_supervisor(state) -> Literal['researcher','coder','writer','__end__']:
    return '__end__' if state['next'] == 'FINISH' else state['next']

graph = StateGraph(SupervisorState)
graph.add_node('supervisor', supervisor_node)
graph.add_node('researcher', researcher_node)
graph.add_node('coder',      coder_node)
graph.add_node('writer',     writer_node)
graph.add_edge(START, 'supervisor')
graph.add_conditional_edges('supervisor', route_supervisor)
for agent in MEMBERS:
    graph.add_edge(agent, 'supervisor')

app = graph.compile()
result = app.invoke({'messages': [HumanMessage('Research and summarise the latest AI news')]})
print(result['messages'][-1].content)
```

READ MORE -- TOPIC REFERENCES

[Docs] LangGraph -- Multi-agent collaboration (no supervisor)
-> https://langchain-ai.github.io/langgraph/tutorials/multi_agent/multi_agent_collaboration/
[Docs] LangGraph -- How to add agent name to messages
-> https://langchain-ai.github.io/langgraph/how-tos/pass_private_state/
[Blog] LangChain -- Hierarchical multi-agent systems
-> https://blog.langchain.dev/langgraph-multi-agent-workflows/

(5) DAY COMPLETION CHECKLIST

- [ ] 3 specialist agents built -- each with appropriate tools and system prompt
- [ ] Supervisor node routes correctly to researcher, coder, or writer
- [ ] FINISH signal works -- supervisor terminates when task is done
- [ ] Multi-step task completed across 2+ agent hops
- [ ] Traced the run -- identified which agents were called and in what order
- [ ] Can explain: supervisor pattern vs peer-to-peer multi-agent collaboration

---

Day 30 . Subgraphs & Complex State -- Reducers & MessagesState

(1) THEORY -- TOPICS TO COVER

Compiling a graph as a subgraph node inside a parent graph
State transformation between parent and subgraph (input/output mapping)
MessagesState -- the built-in state class for message-based agents
add_messages reducer -- appends to messages list instead of replacing it
Writing custom reducers for lists, sets, or counters in state
TypedDict field annotations with Annotated[list, add_messages]

(2) FOCUS / SKIP GUIDE

FOCUS ON

How to compile a graph and then use it as a node: graph.compile() -> parent.add_node()
The state mapping challenge: subgraph may have different fields than parent
add_messages: why returning {'messages': [msg]} appends instead of replacing
Writing a custom reducer: takes (current_value, new_value) -> merged_value
When to use MessagesState vs a custom TypedDict for your state

DON'T WORRY ABOUT YET

Cross-process subgraph communication -- local in-process only for now
Dynamic subgraph creation at runtime -- static subgraphs are fine
Streaming from nested subgraphs with namespace filtering
Subgraph state schema versioning and migration

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] LangGraph -- Subgraphs how-to (official)
-> https://langchain-ai.github.io/langgraph/how-tos/subgraph/
[Docs] LangGraph -- State reducers reference
-> https://langchain-ai.github.io/langgraph/concepts/low_level/#reducers
[Docs] LangGraph -- MessagesState reference
-> https://langchain-ai.github.io/langgraph/reference/graphs/#langgraph.graph.message.MessagesState
[YouTube] LangChain -- LangGraph subgraphs deep dive
-> https://www.youtube.com/watch?v=R8KB-Zcynxc
[Docs] LangGraph -- How to manage agent state
-> https://langchain-ai.github.io/langgraph/how-tos/manage-agent-steps/

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] Build a research_subgraph: search -> summarise, compiled to a node
- [ ] Embed it as a node in a parent graph with add_node('research', research_subgraph)
- [ ] Add state mapping: parent state -> subgraph input, subgraph output -> parent state
- [ ] Add a custom reducer for a 'findings' list that deduplicates items
- [ ] Use Annotated[list, add_messages] in your TypedDict for the messages field
- [ ] Visualise nested graph structure with draw_mermaid()

TASK RESOURCES

[Docs] LangGraph -- Subgraph how-to full example
-> https://langchain-ai.github.io/langgraph/how-tos/subgraph/
[Docs] LangGraph -- Custom reducers how-to
-> https://langchain-ai.github.io/langgraph/how-tos/subgraph-transform-state/
[Docs] LangGraph -- How to transform state in subgraphs
-> https://langchain-ai.github.io/langgraph/how-tos/subgraph-transform-state/

(4) CODE EXAMPLES

Subgraph nested in parent graph

```
from langgraph.graph import StateGraph, START, END, MessagesState
from langchain_core.messages import HumanMessage

# -- Build a subgraph --
class SubState(MessagesState):
    summary: str

def search_node(state: SubState) -> dict:
    query = state['messages'][-1].content
    results = search_tool.invoke(query)
    return {'summary': str(results)[:500]}

def summarise_node(state: SubState) -> dict:
    resp = llm.invoke([HumanMessage(f'Summarise: {state["summary"]}')])
    return {'messages': [resp]}

sub = StateGraph(SubState)
sub.add_node('search',    search_node)
sub.add_node('summarise', summarise_node)
sub.add_edge(START, 'search')
sub.add_edge('search', 'summarise')
sub.add_edge('summarise', END)
research_subgraph = sub.compile()

# -- Embed subgraph in parent --
class ParentState(MessagesState):
    final_answer: str

def finalise_node(state: ParentState) -> dict:
    last = state['messages'][-1].content
    return {'final_answer': f'Answer: {last}'}

parent = StateGraph(ParentState)
parent.add_node('research',  research_subgraph)  # subgraph as node
parent.add_node('finalise',  finalise_node)
parent.add_edge(START,       'research')
parent.add_edge('research',  'finalise')
parent.add_edge('finalise',  END)

app = parent.compile()
result = app.invoke({'messages': [HumanMessage('Latest news on LLMs?')], 'final_answer': ''})
print(result['final_answer'])
```

Custom reducer for deduplication

```
from typing import Annotated
from langgraph.graph.message import add_messages

# Custom reducer: append new items, deduplicate
def append_unique(existing: list, new: list) -> list:
    return list(dict.fromkeys(existing + new))

class ResearchState(MessagesState):
    # messages uses built-in add_messages reducer
    messages: Annotated[list, add_messages]
    # findings uses our custom dedup reducer
    findings: Annotated[list, append_unique]
    # counter uses operator.add reducer
    import operator
    hop_count: Annotated[int, operator.add]

def agent_node(state: ResearchState) -> dict:
    return {
        'findings': ['new finding from this hop'],
        'hop_count': 1,  # reducer adds this to current count
    }
```

READ MORE -- TOPIC REFERENCES

[Docs] LangGraph -- How to stream from subgraphs
-> https://langchain-ai.github.io/langgraph/how-tos/streaming-subgraphs/
[Docs] LangGraph -- How to view and update subgraph state
-> https://langchain-ai.github.io/langgraph/how-tos/subgraphs-manage-state/

(5) DAY COMPLETION CHECKLIST

- [ ] Research subgraph built and compiled -- runs independently
- [ ] Subgraph embedded as a node in parent graph -- parent runs end-to-end
- [ ] State mapping works -- data flows correctly between parent and subgraph
- [ ] Custom dedup reducer built and tested -- no duplicates in findings list
- [ ] Annotated[list, add_messages] used for messages field in TypedDict
- [ ] Nested graph visualised with draw_mermaid() -- structure is clear
- [ ] Can explain: when to use MessagesState vs a custom TypedDict

---

Day 31 . Parallelism in LangGraph -- Fan-Out & Fan-In

(1) THEORY -- TOPICS TO COVER

Fan-out: one node connects to multiple downstream nodes (runs in parallel)
Fan-in: multiple nodes converge into one aggregation node
LangGraph runs parallel branches concurrently when no data dependency exists
Aggregating parallel results with reducers in shared state
Map-reduce pattern in LangGraph: dynamic fan-out over a list of items
Send() API -- dynamically spawning parallel tasks at runtime

(2) FOCUS / SKIP GUIDE

FOCUS ON

Fan-out syntax: add_edge(A, B) AND add_edge(A, C) -- B and C run in parallel
Fan-in: both B and C must have add_edge(B, D) and add_edge(C, D) to converge
Results are merged via reducers -- append_unique or add for numeric fields
Measuring the speedup: time parallel vs sequential with the same 3 agents
Send() for dynamic map: iterate over a list and spawn one node per item

DON'T WORRY ABOUT YET

Async parallelism with asyncio -- LangGraph handles concurrency internally
Cross-machine distributed fan-out -- in-process only for now
Load balancing across parallel branches -- not needed at this scale
Dynamic graph rewriting at runtime beyond Send()

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] LangGraph -- Parallel branching how-to
-> https://langchain-ai.github.io/langgraph/how-tos/branching/
[Docs] LangGraph -- Map-reduce how-to
-> https://langchain-ai.github.io/langgraph/how-tos/map-reduce/
[Docs] LangGraph -- Send() API reference
-> https://langchain-ai.github.io/langgraph/concepts/low_level/#send
[YouTube] LangChain -- LangGraph parallel execution
-> https://www.youtube.com/watch?v=R0XGQHL-4LM
[Docs] LangGraph -- How to create branches (fork)
-> https://langchain-ai.github.io/langgraph/how-tos/branching/

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] Build fan-out graph: one input -> 3 agents running in parallel
- [ ] Add fan-in synthesis node: receives all 3 results and combines them
- [ ] Measure parallel execution time vs sequential -- print both
- [ ] Build map-reduce: split a document list -> parallel summarise -> combine
- [ ] Implement Send() API to dynamically fan-out over a variable-length list

TASK RESOURCES

[Docs] LangGraph -- Parallel node execution reference
-> https://langchain-ai.github.io/langgraph/how-tos/branching/
[Docs] LangGraph -- Send API how-to (map-reduce)
-> https://langchain-ai.github.io/langgraph/how-tos/map-reduce/

(4) CODE EXAMPLES

Fan-out + fan-in parallel execution

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
import operator, time

class ParallelState(TypedDict):
    input:   str
    results: Annotated[list, operator.add]  # reducer appends

def agent_a(state): return {'results': [f'Agent A: {state["input"][:30]}...']}
def agent_b(state): return {'results': [f'Agent B: {state["input"][:30]}...']}
def agent_c(state): return {'results': [f'Agent C: {state["input"][:30]}...']}

def synthesise(state):
    combined = ' | '.join(state['results'])
    return {'results': [f'SYNTHESIS: {combined}']}

graph = StateGraph(ParallelState)
graph.add_node('agent_a',   agent_a)
graph.add_node('agent_b',   agent_b)
graph.add_node('agent_c',   agent_c)
graph.add_node('synthesise',synthesise)

# Fan-out: START -> all 3 agents in parallel
graph.add_edge(START,     'agent_a')
graph.add_edge(START,     'agent_b')
graph.add_edge(START,     'agent_c')

# Fan-in: all 3 agents -> synthesise
graph.add_edge('agent_a', 'synthesise')
graph.add_edge('agent_b', 'synthesise')
graph.add_edge('agent_c', 'synthesise')
graph.add_edge('synthesise', END)

app   = graph.compile()
start = time.time()
result= app.invoke({'input': 'Analyse AI trends', 'results': []})
print(f'Parallel took: {time.time()-start:.2f}s')
print(result['results'][-1])
```

Send() API -- dynamic map-reduce

```
from langgraph.types import Send
from typing import TypedDict, Annotated
import operator

class MapState(TypedDict):
    documents: list[str]
    summaries: Annotated[list, operator.add]
    final:     str

class WorkerState(TypedDict):
    document: str
    summaries: Annotated[list, operator.add]

# Map: fan-out -- one worker per document
def map_node(state: MapState):
    return [Send('summarise_worker', {'document': doc, 'summaries': []}) for doc in state['documents']]

def summarise_worker(state: WorkerState) -> dict:
    resp = llm.invoke([HumanMessage(f'Summarise in 1 sentence: {state["document"]}')])
    return {'summaries': [resp.content]}

# Reduce: combine all summaries
def reduce_node(state: MapState) -> dict:
    combined = chr(10).join(state['summaries'])
    resp = llm.invoke([HumanMessage(f'Combine these summaries: {combined}')])
    return {'final': resp.content}

graph = StateGraph(MapState)
graph.add_node('map',    map_node)
graph.add_node('summarise_worker', summarise_worker)
graph.add_node('reduce', reduce_node)
graph.add_conditional_edges('map', lambda s: s, ['summarise_worker'])
graph.add_edge(START, 'map')
graph.add_edge('summarise_worker', 'reduce')
graph.add_edge('reduce', END)
```

READ MORE -- TOPIC REFERENCES

[Docs] LangGraph -- How to create map-reduce branches
-> https://langchain-ai.github.io/langgraph/how-tos/map-reduce/
[Docs] LangGraph -- Conditional branching reference
-> https://langchain-ai.github.io/langgraph/how-tos/branching/

(5) DAY COMPLETION CHECKLIST

- [ ] Fan-out graph: input -> 3 agents all run concurrently
- [ ] Fan-in: all 3 results collected and synthesised in one node
- [ ] Parallel vs sequential timing measured -- parallel is faster
- [ ] Map-reduce with Send() dynamically fans out over a list of documents
- [ ] Custom operator.add reducer aggregates list results correctly
- [ ] Can explain: how add_edge creates parallelism vs sequential edges

Day 32 . Error Handling & Retries in LangGraph

(1) THEORY -- TOPICS TO COVER

retry_policy on graph nodes -- automatic retry with backoff on failure
Fallback nodes: add_conditional_edges routes to error handler on exception
GraphRecursionError -- what causes it and how to set recursion_limit
try/except inside node functions vs graph-level retry policies
Logging and debugging failed runs with LangSmith trace inspection
Graceful degradation: returning partial results instead of crashing

(2) FOCUS / SKIP GUIDE

FOCUS ON

RetryPolicy: max_attempts and retry_on (which exceptions trigger retry)
The error handler node pattern: catches exceptions, logs, returns safe fallback
recursion_limit=25 in graph.compile() -- prevents agent infinite loops
Inspecting a failed LangSmith run: finding the exact node and error message
The difference between transient errors (retry) vs logic errors (fallback)

DON'T WORRY ABOUT YET

Distributed circuit breakers across multiple services -- local graphs only
Dead letter queues and async error recovery pipelines
Custom retry schedulers with exponential backoff from scratch
Error aggregation dashboards -- LangSmith covers this for development

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] LangGraph -- Error handling guide
-> https://langchain-ai.github.io/langgraph/how-tos/node-retries/
[Docs] LangGraph -- How to add node retry policies
-> https://langchain-ai.github.io/langgraph/how-tos/node-retries/
[Docs] LangSmith -- Inspecting failed runs
-> https://docs.smith.langchain.com/
[YouTube] LangChain -- Production error handling
-> https://www.youtube.com/watch?v=hvAPnpSfSGo
[Docs] LangGraph -- recursion_limit reference
-> https://langchain-ai.github.io/langgraph/reference/graphs/

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] Add retry_policy to a node that calls an external API (e.g. Tavily)
- [ ] Simulate a flaky tool -- fails 50% of the time, verify retries kick in
- [ ] Add an error_handler node -- routes there when any node raises an exception
- [ ] Set recursion_limit=10 on your Day 29 supervisor -- trigger it intentionally
- [ ] Use LangSmith to inspect a failed run -- find the failing node and traceback
- [ ] Wrap a node in try/except to return a graceful fallback instead of crashing

TASK RESOURCES

[Docs] LangGraph -- RetryPolicy API reference
-> https://langchain-ai.github.io/langgraph/reference/graphs/#langgraph.graph.graph.CompiledGraph
[Docs] LangGraph -- GraphRecursionError docs
-> https://langchain-ai.github.io/langgraph/reference/errors/
[Docs] LangSmith -- Filtering runs by error
-> https://docs.smith.langchain.com/how_to_guides/monitoring/filter_traces_in_application

(4) CODE EXAMPLES

RetryPolicy + fallback error handler

```
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.pregel import RetryPolicy
from langchain_core.messages import HumanMessage, AIMessage
import random

# Flaky node -- fails 50% of the time
def flaky_search_node(state: MessagesState) -> dict:
    if random.random() < 0.5:
        raise ConnectionError('Search API timeout -- retrying...')
    return {'messages': [AIMessage('Search result: found relevant data')]}

# Error handler node -- graceful fallback
def error_handler_node(state: MessagesState) -> dict:
    print('ERROR: node failed after retries -- returning fallback')
    return {'messages': [AIMessage('Sorry, search unavailable. Using cached data.')]}

def final_node(state: MessagesState) -> dict:
    return {'messages': [AIMessage(f'Final answer based on: {state["messages"][-1].content}')]}

graph = StateGraph(MessagesState)

# Attach retry policy directly to the node
graph.add_node('search', flaky_search_node,
    retry=RetryPolicy(max_attempts=3, retry_on=(ConnectionError,)))
graph.add_node('error_handler', error_handler_node)
graph.add_node('final', final_node)

graph.add_edge(START, 'search')
graph.add_edge('search', 'final')
graph.add_edge('error_handler', 'final')
graph.add_edge('final', END)

# Compile with recursion limit
app = graph.compile()
result = app.invoke({'messages': [HumanMessage('Find info about LLMs')]},
    config={'recursion_limit': 25})
print(result['messages'][-1].content)
```

Graceful fallback with try/except inside node

```
def safe_tool_node(state: MessagesState) -> dict:
    try:
        result = expensive_api_call(state['messages'][-1].content)
        return {'messages': [AIMessage(result)]}
    except TimeoutError:
        print('WARNING: API timeout -- using cached fallback')
        return {'messages': [AIMessage('Using cached data: [fallback result]')]}
    except Exception as e:
        print(f'ERROR: unexpected failure: {e}')
        return {'messages': [AIMessage(f'Error occurred: {str(e)[:100]}')]}
```

READ MORE -- TOPIC REFERENCES

[Docs] LangGraph -- All error types reference
-> https://langchain-ai.github.io/langgraph/reference/errors/
[Blog] LangChain -- Building reliable agents
-> https://blog.langchain.dev/reliability-in-agents/

(5) DAY COMPLETION CHECKLIST

- [ ] RetryPolicy added to a flaky node -- retries visible in logs
- [ ] Flaky tool simulated -- verified retries fire before failure
- [ ] Error handler node routes correctly when node fails after all retries
- [ ] recursion_limit set -- GraphRecursionError triggered intentionally and caught
- [ ] Failed run inspected in LangSmith -- found the exact failing node
- [ ] try/except fallback works -- node returns safe response instead of crashing
- [ ] Can explain: retry_policy vs try/except -- when to use each approach

---

Day 33 . LangGraph Studio & Visualisation

(1) THEORY -- TOPICS TO COVER

LangGraph Studio -- live graph visualisation and step-through debugging
draw_mermaid() -- exporting graph as Mermaid diagram code
get_graph().print_ascii() -- quick terminal diagram for any graph
LangSmith traces for multi-agent runs -- navigating the nested spans
Debugging strategies: stream_mode='debug' for full event trace

(2) FOCUS / SKIP GUIDE

FOCUS ON

draw_mermaid() always works -- use mermaid.live if Studio install fails
Reading a multi-agent LangSmith trace: each sub-agent is a nested span
stream_mode='debug' -- the most verbose output, shows every event
Using print_ascii() during development to quickly check graph structure
Mermaid PNG export workflow: code -> mermaid.live -> download PNG

DON'T WORRY ABOUT YET

LangGraph Cloud Studio (cloud-hosted) -- focus on local for now
Custom Studio plugins or extensions
Automated diagram CI/CD -- generating diagrams on every commit
Grafana or Prometheus integration for production monitoring

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] LangGraph Studio install guide
-> https://github.com/langchain-ai/langgraph-studio
[Docs] LangGraph Studio docs (official)
-> https://langchain-ai.github.io/langgraph/concepts/langgraph_studio/
[YouTube] LangChain -- LangGraph Studio demo
-> https://www.youtube.com/watch?v=3Y4dBqsNOxs
[Tool] Mermaid Live Editor
-> https://mermaid.live
[Docs] LangGraph -- stream_mode='debug' reference
-> https://langchain-ai.github.io/langgraph/how-tos/streaming/

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] Export Mermaid diagram of your Day 29 supervisor graph -- paste into mermaid.live
- [ ] Export Mermaid diagram of your Day 31 parallel graph -- compare structures
- [ ] Run a multi-agent task with stream_mode='debug' -- read all events printed
- [ ] Open LangSmith and find a multi-agent run -- expand nested sub-agent spans
- [ ] Try installing LangGraph Studio (optional -- use Mermaid export if it fails)
- [ ] Save all diagram PNGs and write a short note on each graph's structure

TASK RESOURCES

[Docs] LangGraph -- draw_mermaid reference
-> https://langchain-ai.github.io/langgraph/how-tos/draw-graphs/
[Docs] LangGraph -- streaming debug mode
-> https://langchain-ai.github.io/langgraph/how-tos/streaming/
[Docs] LangSmith -- Navigating nested traces
-> https://docs.smith.langchain.com/

(4) CODE EXAMPLES

Visualisation toolkit

```
# 1. ASCII diagram in terminal (always works)
app.get_graph().print_ascii()

# 2. Mermaid code (paste at https://mermaid.live)
mermaid = app.get_graph().draw_mermaid()
print(mermaid)

# 3. Save Mermaid to file
with open('graph_diagram.md', 'w') as f:
    f.write('```mermaid\n' + mermaid + '\n```')

# 4. For nested subgraphs -- use xray=True
app.get_graph(xray=True).draw_mermaid_png(output_file_path='graph.png')
# Requires: pip install grandalf

# 5. Stream with debug mode -- most verbose
for event in app.stream(
    {'messages': [HumanMessage('Test query')]},
    stream_mode='debug'
):
    print(f'Event type: {event["type"]} | node: {event.get("step")}')
```

LangSmith multi-agent trace analysis

```
# Ensure LangSmith tracing is on
import os
os.environ['LANGCHAIN_TRACING_V2'] = 'true'
os.environ['LANGCHAIN_PROJECT']    = 'week5-multiagent'

# Run your multi-agent graph
result = app.invoke({'messages': [HumanMessage('Research quantum computing')]})

# In LangSmith UI:
# 1. Open smith.langchain.com -> week5-multiagent project
# 2. Find the run -- click to expand
# 3. Look for nested spans: supervisor -> researcher -> summariser
# 4. Click each span to see the exact prompt + response
# 5. Check 'Latency' column -- find the slowest agent
# 6. Check 'Tokens' column -- find the most expensive call
```

READ MORE -- TOPIC REFERENCES

[Docs] LangGraph -- How to stream subgraph events
-> https://langchain-ai.github.io/langgraph/how-tos/streaming-subgraphs/
[Docs] LangSmith -- Multi-agent trace docs
-> https://docs.smith.langchain.com/

(5) DAY COMPLETION CHECKLIST

- [ ] Mermaid diagram exported for both supervisor graph and parallel graph
- [ ] Both diagrams pasted into mermaid.live -- structures are clear and correct
- [ ] stream_mode='debug' run -- read and understood the event types printed
- [ ] Multi-agent LangSmith trace found -- nested sub-agent spans expanded
- [ ] Slowest agent and most expensive call identified in LangSmith
- [ ] All diagram PNGs saved with notes on each graph structure

- [ ] Can explain: what information stream_mode='debug' gives that 'updates' does not

Day 34-35 . Week 5 Project -- Multi-Agent Research System

(1) THEORY -- TOPICS TO COVER

LangGraph async execution: ainvoke, astream, astream_events patterns
asyncio.run() -- running async graph from synchronous Python
Async tool calls: make tool functions async for IO-bound operations
LangGraph Platform overview -- what deployment looks like in production

(2) FOCUS / SKIP GUIDE

FOCUS ON

Building the full 3-agent pipeline: searcher -> summariser -> fact_checker
Adding checkpointing + human approval before the final output is returned
Making the system async -- astream_events for real-time token streaming
Tracing the full run in LangSmith and identifying the slowest agent
Putting all Week 5 skills together into one working research system

DON'T WORRY ABOUT YET

Deploying to LangGraph Cloud Platform -- local async is the goal here
Auto-scaling and load balancing multiple research requests
Fine-tuning the supervisor routing for production quality

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] LangGraph -- Async how-to guide
-> https://langchain-ai.github.io/langgraph/how-tos/async/
[Docs] LangGraph -- How to stream async events
-> https://langchain-ai.github.io/langgraph/how-tos/astream_events/
[YouTube] LangChain -- Multi-agent research workflow tutorial
-> https://www.youtube.com/watch?v=hvAPnpSfSGo
[Docs] LangGraph Platform overview
-> https://langchain-ai.github.io/langgraph/concepts/langgraph_platform/

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] Build supervisor -> web_searcher, summariser, fact_checker pipeline
- [ ] Searcher finds 3 URLs; summariser condenses; fact_checker validates claims
- [ ] Add SqliteSaver checkpointing + interrupt_before fact_checker output
- [ ] Convert all nodes to async functions and use astream
- [ ] Stream token-by-token output from the summariser node
- [ ] Trace the full run in LangSmith -- identify the slowest agent
- [ ] Test with 3 different research topics -- evaluate output quality

TASK RESOURCES

[Docs] LangGraph -- async node functions
-> https://langchain-ai.github.io/langgraph/how-tos/async/
[Docs] LangGraph -- astream_events reference
-> https://langchain-ai.github.io/langgraph/how-tos/astream_events/
[Docs] LangGraph -- streaming tokens from LLM
-> https://langchain-ai.github.io/langgraph/how-tos/streaming-tokens/

(4) CODE EXAMPLES

Async multi-agent research system

```
import asyncio
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.checkpoint.sqlite.aio import AsyncSqliteSaver
from langchain_core.messages import HumanMessage, AIMessage

# Async specialist agents
async def searcher_node(state: MessagesState) -> dict:
    query   = state['messages'][-1].content
    results = await search_tool.ainvoke(query)  # async Tavily call
    return {'messages': [AIMessage(f'Found: {str(results)[:300]}')]}

async def summariser_node(state: MessagesState) -> dict:
    content = state['messages'][-1].content
    resp = await llm.ainvoke([HumanMessage(f'Summarise concisely: {content}')])
    return {'messages': [resp]}

async def fact_checker_node(state: MessagesState) -> dict:
    summary = state['messages'][-1].content
    resp = await llm.ainvoke([HumanMessage(f'Fact-check this summary: {summary}')])
    return {'messages': [resp]}

# Build graph
graph = StateGraph(MessagesState)
graph.add_node('searcher',    searcher_node)
graph.add_node('summariser',  summariser_node)
graph.add_node('fact_checker',fact_checker_node)
graph.add_edge(START,         'searcher')
graph.add_edge('searcher',    'summariser')
graph.add_edge('summariser',  'fact_checker')
graph.add_edge('fact_checker', END)

async def run():
    async with AsyncSqliteSaver.from_conn_string('research.db') as ckpt:
        app = graph.compile(checkpointer=ckpt, interrupt_before=['fact_checker'])
        cfg = {'configurable': {'thread_id': 'research_1'}}
        # Stream events as they happen
        async for event in app.astream_events(
            {'messages': [HumanMessage('Latest breakthroughs in quantum computing')]},
            config=cfg, version='v2'
        ):
            if event['event'] == 'on_chat_model_stream':
                token = event['data']['chunk'].content
                if token: print(token, end='', flush=True)

asyncio.run(run())
```

READ MORE -- TOPIC REFERENCES

[Docs] LangGraph -- AsyncSqliteSaver reference
-> https://langchain-ai.github.io/langgraph/reference/checkpoints/
[Blog] LangChain -- Production multi-agent systems
-> https://blog.langchain.dev/langgraph-multi-agent-workflows/

(5) DAY COMPLETION CHECKLIST

- [ ] 3-agent pipeline built: searcher -> summariser -> fact_checker
- [ ] Each agent returns meaningful output from its specialist role
- [ ] SqliteSaver checkpointing + interrupt_before fact_checker working
- [ ] All nodes converted to async functions -- ainvoke used for LLM calls
- [ ] astream_events streaming: tokens print in real time from summariser
- [ ] LangSmith trace shows full multi-agent run -- slowest agent identified
- [ ] 3 research topics tested -- output quality reviewed and documented
- [ ] WEEK 5 COMPLETE -- ready for Week 6 real projects

Week 6 -- Real Projects

Day 36-38 . Project 1 -- AI Customer Support Bot

(1) THEORY -- TOPICS TO COVER

Conversational AI design patterns for multi-turn support flows
Intent classification and routing: order / FAQ / complaint / escalation
LangGraph prebuilt create_react_agent helper for each support role
Human escalation with interrupt_before -- passing to a live agent
Conversation persistence: SqliteSaver for full history per customer

(2) FOCUS / SKIP GUIDE

FOCUS ON

The 4-intent router: classify incoming message -> route to specialist node
Tools: lookup_order(order_id), search_faq(query), escalate_to_human(reason)
Persisting every conversation by customer_id thread -- full history available
interrupt_before escalation -- human agent reviews before the bot escalates
Testing with 20 realistic messages covering all 4 intent types

DON'T WORRY ABOUT YET

Connecting to a real ticketing system (Zendesk, Salesforce) -- mock tools are fine
Multilingual support and translation -- English only for now
Real-time human agent handoff via websockets -- simulate with CLI input
Analytics dashboard for support metrics -- LangSmith covers this

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] LangGraph customer support tutorial (official)
-> https://langchain-ai.github.io/langgraph/tutorials/customer-support/customer-support/
[YouTube] Alejandro AO -- Customer support AI with LangGraph
-> https://www.youtube.com/watch?v=Hx8jwab5Xt4
[Blog] LangChain -- Building a support bot with LangGraph
-> https://blog.langchain.dev/customer-support-with-langgraph/
[Docs] LangGraph -- create_react_agent prebuilt
-> https://langchain-ai.github.io/langgraph/reference/prebuilt/

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] Define 3 mock tools: lookup_order(), search_faq(), escalate_to_human()
- [ ] Build intent classifier node: routes to order/faq/complaint/escalation
- [ ] Build 3 specialist nodes: order_agent, faq_agent, complaint_agent
- [ ] Add SqliteSaver -- persist each customer conversation by customer_id
- [ ] Add interrupt_before=['escalation'] -- human reviews before escalating
- [ ] Test with 20 realistic customer messages across all 4 intent types
- [ ] Trace a full conversation in LangSmith -- verify correct routing

TASK RESOURCES

[Docs] LangGraph -- customer support full tutorial
-> https://langchain-ai.github.io/langgraph/tutorials/customer-support/customer-support/
[Docs] LangGraph -- How to add semantic routing
-> https://python.langchain.com/docs/how_to/routing/
[Docs] LangSmith -- Filtering traces by project
-> https://docs.smith.langchain.com/how_to_guides/monitoring/filter_traces_in_application

(4) CODE EXAMPLES

Customer support bot structure

```
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.types import Command
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, AIMessage
from typing import Literal

@tool
def lookup_order(order_id: str) -> str:
    '''Look up the status of a customer order by ID.'''
    orders = {'ORD001': 'Shipped - arrives Thursday', 'ORD002': 'Processing'}
    return orders.get(order_id, f'Order {order_id} not found')

@tool
def search_faq(query: str) -> str:
    '''Search the FAQ for answers to common questions.'''
    return f'FAQ result for "{query}": [answer would appear here]'

@tool
def escalate_to_human(reason: str) -> str:
    '''Escalate this conversation to a human agent.'''
    return f'Escalation requested: {reason}. A human agent will contact you shortly.'

# Intent classifier
def classify_intent(state: MessagesState) -> dict:
    msg = state['messages'][-1].content.lower()
    if any(w in msg for w in ['order','track','ship','delivery']):
        return {'intent': 'order'}
    elif any(w in msg for w in ['how','what','policy','return','refund']):
        return {'intent': 'faq'}
    elif any(w in msg for w in ['angry','unacceptable','manager','complaint']):
        return {'intent': 'complaint'}
    return {'intent': 'general'}
```

Support bot -- graph assembly & run

```
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages
class SupportState(MessagesState):
    intent: str

def route_intent(state) -> Literal['order_agent','faq_agent','complaint_agent','general_agent']:
    return f'{state["intent"]}_agent'

from langgraph.prebuilt import create_react_agent
order_agent     = create_react_agent(llm, tools=[lookup_order], state_schema=MessagesState)
faq_agent       = create_react_agent(llm, tools=[search_faq],   state_schema=MessagesState)
complaint_agent = create_react_agent(llm, tools=[escalate_to_human], state_schema=MessagesState)
general_agent   = create_react_agent(llm, tools=[],              state_schema=MessagesState)

graph = StateGraph(SupportState)
graph.add_node('classify',        classify_intent)
graph.add_node('order_agent',     lambda s: order_agent.invoke(s))
graph.add_node('faq_agent',       lambda s: faq_agent.invoke(s))
graph.add_node('complaint_agent', lambda s: complaint_agent.invoke(s))
graph.add_node('general_agent',   lambda s: general_agent.invoke(s))
graph.add_edge(START, 'classify')
graph.add_conditional_edges('classify', route_intent)
for node in ['order_agent','faq_agent','complaint_agent','general_agent']:
    graph.add_edge(node, END)

with SqliteSaver.from_conn_string('support.db') as ckpt:
    app = graph.compile(checkpointer=ckpt)
    cfg = {'configurable': {'thread_id': 'customer_42'}}
    resp = app.invoke({'messages': [HumanMessage('Where is my order ORD001?')], 'intent': ''}, config=cfg)
    print(resp['messages'][-1].content)
```

READ MORE -- TOPIC REFERENCES

[Docs] LangGraph -- Full customer support tutorial with flight booking
-> https://langchain-ai.github.io/langgraph/tutorials/customer-support/customer-support/
[Blog] LangChain -- Best practices for support bots
-> https://blog.langchain.dev/customer-support-with-langgraph/

(5) DAY COMPLETION CHECKLIST

- [ ] 3 mock tools built: lookup_order, search_faq, escalate_to_human
- [ ] Intent classifier routes correctly across all 4 intent types
- [ ] 4 specialist agent nodes built and connected via conditional routing
- [ ] SqliteSaver persists conversations -- customer history survives restarts
- [ ] interrupt_before escalation -- human approval step tested
- [ ] 20 test messages sent across all intent types -- all routed correctly
- [ ] LangSmith trace verified -- routing decisions visible for each message

---

Day 39-42 . Project 2 -- Coding Assistant Agent

(1) THEORY -- TOPICS TO COVER

Plan-and-execute pattern: plan all steps first, then execute each step
Code execution safety basics: subprocess with timeout and output limits
Few-shot prompting for reliable code generation -- examples in system prompt
Self-reflection loop: agent reviews and fixes its own code output
LangSmith evaluation with a code correctness rubric

(2) FOCUS / SKIP GUIDE

FOCUS ON

The plan node: LLM breaks the task into numbered steps before any code is written
The execute node: LLM writes code for one step at a time, following the plan
The reflect node: LLM reads the code + output and decides pass or retry
subprocess safety: timeout=10, capture_output=True, never shell=True with user input
Few-shot examples in the system prompt -- 2-3 examples of good code format

DON'T WORRY ABOUT YET

Docker/container sandboxing for code execution -- subprocess with timeout is fine
Handling all edge cases in code execution (malicious input, etc.) -- toy environment
Multi-language support beyond Python -- Python only
Code diffing and patch-based editing -- full rewrites are fine for this project

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] LangGraph code assistant tutorial (official)
-> https://langchain-ai.github.io/langgraph/tutorials/code_assistant/langgraph_code_assistant/
[YouTube] LangChain -- AI code assistant with LangGraph
-> https://www.youtube.com/watch?v=MvNdgmM7uyc
[Docs] LangGraph -- Plan-and-execute agent how-to
-> https://langchain-ai.github.io/langgraph/tutorials/plan-and-execute/plan-and-execute/
[Blog] LangChain -- Self-correcting code agent
-> https://blog.langchain.dev/code-execution-with-langgraph/

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] Build plan node: LLM outputs a numbered list of steps for the coding task
- [ ] Build execute node: LLM writes Python code for the current step
- [ ] Build run_code node: executes the code with subprocess, captures output
- [ ] Build reflect node: LLM reviews code + output, returns pass or retry + fix
- [ ] Add loop: reflect -> execute if retry, reflect -> next_step if pass
- [ ] Add few-shot examples to the code generation system prompt
- [ ] Test on 10 real coding tasks -- log pass/fail and number of retries per task
- [ ] Add LangSmith evaluation with a code correctness rubric (LLM-as-judge)

TASK RESOURCES

[Docs] LangGraph -- Plan-and-execute tutorial
-> https://langchain-ai.github.io/langgraph/tutorials/plan-and-execute/plan-and-execute/
[Docs] Python docs -- subprocess module
-> https://docs.python.org/3/library/subprocess.html
[Docs] LangSmith -- Custom evaluators for code
-> https://docs.smith.langchain.com/how_to_guides/evaluation/evaluate_llm_application

(4) CODE EXAMPLES

Plan-Execute-Reflect coding agent

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Literal
from langchain_core.messages import HumanMessage
import subprocess

class CodeState(TypedDict):
    task:        str
    plan:        list[str]
    current_step:int
    code:        str
    output:      str
    retries:     int
    status:      str  # 'pass' or 'retry'

# Plan node
def plan_node(state: CodeState) -> dict:
    resp = llm.invoke([HumanMessage(
        f'Break this coding task into 3-5 numbered steps: {state["task"]}'
    )])
    steps = [l.strip() for l in resp.content.split('\n') if l.strip() and l[0].isdigit()]
    return {'plan': steps, 'current_step': 0}

# Execute node
def execute_node(state: CodeState) -> dict:
    step = state['plan'][state['current_step']]
    resp = llm.invoke([HumanMessage(
        f'Write Python code for this step only: {step}\n'
        f'Return ONLY the code, no explanation, no markdown.'
    )])
    code = resp.content.strip().replace('```python','').replace('```','')
    return {'code': code}

# Run code node -- subprocess with safety limits
def run_code_node(state: CodeState) -> dict:
    try:
        result = subprocess.run(
            ['python3', '-c', state['code']],
            capture_output=True, text=True, timeout=10
        )
        output = result.stdout[:500] if result.returncode == 0 else f'ERROR: {result.stderr[:300]}'
    except subprocess.TimeoutExpired:
        output = 'ERROR: Code execution timed out (10s limit)'
    return {'output': output}

# Reflect node
def reflect_node(state: CodeState) -> dict:
    resp = llm.invoke([HumanMessage(
        f'Code: {state["code"]}\nOutput: {state["output"]}\n'
        f'Did this code work correctly? Reply PASS or RETRY: <explanation>'
    )])
    status = 'pass' if resp.content.upper().startswith('PASS') else 'retry'
    return {'status': status, 'retries': state.get('retries', 0) + (1 if status=='retry' else 0)}
```

Coding agent -- routing, graph wiring & run

```
def route_reflect(state: CodeState) -> Literal['execute', 'next_step', '__end__']:
    if state['status'] == 'retry' and state['retries'] < 3:
        return 'execute'
    next_step = state['current_step'] + 1
    if next_step >= len(state['plan']):
        return '__end__'
    return 'next_step'

def next_step_node(state: CodeState) -> dict:
    return {'current_step': state['current_step'] + 1, 'retries': 0}

graph = StateGraph(CodeState)
graph.add_node('plan',      plan_node)
graph.add_node('execute',   execute_node)
graph.add_node('run_code',  run_code_node)
graph.add_node('reflect',   reflect_node)
graph.add_node('next_step', next_step_node)
graph.add_edge(START,        'plan')
graph.add_edge('plan',       'execute')
graph.add_edge('execute',    'run_code')
graph.add_edge('run_code',   'reflect')
graph.add_conditional_edges('reflect', route_reflect)
graph.add_edge('next_step',  'execute')

app = graph.compile()
result = app.invoke({'task': 'Write a function to find all prime numbers up to N using the Sieve of Eratosth
    'plan':[],'current_step':0,'code':'','output':'','retries':0,'status':''})
print('Final code:', result['code'])
print('Output:', result['output'])
```

LangSmith evaluation with code rubric

```
from langsmith import Client
from langsmith.evaluation import evaluate

client = Client()

# Create evaluation dataset
coding_tasks = [
    {'inputs': {'task': 'Sort a list of numbers'}, 'outputs': {'expected': 'sorted list'}},
    {'inputs': {'task': 'Find fibonacci(10)'},     'outputs': {'expected': '55'}},
    {'inputs': {'task': 'Reverse a string'},       'outputs': {'expected': 'reversed string'}},
]
dataset = client.create_dataset('coding-agent-eval')
client.create_examples(dataset_id=dataset.id, examples=coding_tasks)

# Evaluator: LLM-as-judge for code correctness
def code_correctness_evaluator(run, example):
    output  = run.outputs.get('output', '')
    task    = example.inputs['task']
    resp    = llm.invoke([HumanMessage(
        f'Task: {task}\nCode output: {output}\n'
        f'Did the code correctly solve the task? Score 1 (yes) or 0 (no). Reply ONLY with 0 or 1.'
    )])
    score = 1 if '1' in resp.content else 0
    return {'key': 'correctness', 'score': score}

def run_agent(inputs):
    result = app.invoke({**inputs, 'plan':[],'current_step':0,'code':'','output':'','retries':0,'status':''})
    return {'output': result['output'], 'code': result['code']}

results = evaluate(run_agent, data='coding-agent-eval', evaluators=[code_correctness_evaluator])
print(results.to_pandas()[['inputs','outputs','feedback.correctness']])
```

READ MORE -- TOPIC REFERENCES

[Docs] LangGraph -- Code generation with self-correction tutorial
-> https://langchain-ai.github.io/langgraph/tutorials/code_assistant/langgraph_code_assistant/
[Docs] LangGraph -- Plan-and-execute agent tutorial
-> https://langchain-ai.github.io/langgraph/tutorials/plan-and-execute/plan-and-execute/
[Blog] LangChain -- AlphaCode-style coding agent
-> https://blog.langchain.dev/code-execution-with-langgraph/

(5) DAY COMPLETION CHECKLIST

- [ ] Plan node outputs a numbered list of steps before any code is written
- [ ] Execute node generates Python code for the current step
- [ ] run_code node executes with subprocess, timeout=10 -- output captured
- [ ] Reflect node returns PASS or RETRY with explanation
- [ ] Retry loop works: RETRY -> re-execute, PASS -> advance to next step
- [ ] Few-shot examples in code generation system prompt -- output format is consistent
- [ ] 10 coding tasks tested -- pass rate and avg retries per task logged
- [ ] LangSmith evaluation ran -- correctness scores visible per task
- [ ] WEEK 6 COMPLETE -- real projects built, ready for Week 7 production